
Pointer Networks

Oriol Vinyals*
Google Brain

Meire Fortunato*
Department of Mathematics, UC Berkeley

Navdeep Jaitly
Google Brain

Abstract

We introduce a new neural architecture to learn the conditional probability of an output sequence with elements that are discrete tokens corresponding to positions in an input sequence. Such problems cannot be trivially addressed by existent approaches such as sequence-to-sequence [1] and Neural Turing Machines [2], because the number of target classes in each step of the output depends on the length of the input, which is variable. Problems such as sorting variable sized sequences, and various combinatorial optimization problems belong to this class. Our model solves the problem of variable size output dictionaries using a recently proposed mechanism of neural attention. It differs from the previous attention attempts in that, instead of using attention to blend hidden units of an encoder to a context vector at each decoder step, it uses attention as a pointer to select a member of the input sequence as the output. We call this architecture a Pointer Net (Ptr-Net). We show Ptr-Nets can be used to learn approximate solutions to three challenging geometric problems – finding planar convex hulls, computing Delaunay triangulations, and the planar Travelling Salesman Problem – using training examples alone. Ptr-Nets not only improve over sequence-to-sequence with input attention, but also allow us to generalize to variable size output dictionaries. We show that the learnt models generalize beyond the maximum lengths they were trained on. We hope our results on these tasks will encourage a broader exploration of neural learning for discrete problems.

1 Introduction

Recurrent Neural Networks (RNNs) have been used for learning functions over sequences from examples for more than three decades [3]. However, their architecture limited them to settings where the inputs and outputs were available at a fixed frame rate (e.g. [4]). The recently introduced sequence-to-sequence paradigm [1] removed these constraints by using one RNN to map an input sequence to an embedding and another (possibly the same) RNN to map the embedding to an output sequence. Bahdanau et. al. augmented the decoder by propagating extra contextual information from the input using a content-based attentional mechanism [5, 2, 6]. These developments have made it possible to apply RNNs to new domains, achieving state-of-the-art results in core problems in natural language processing such as translation [1, 5] and parsing [7], image and video captioning [8, 9], and even learning to execute small programs [2, 10].

Nonetheless, these methods still require the size of the output dictionary to be fixed *a priori*. Because of this constraint we cannot directly apply this framework to combinatorial problems where the size of the output dictionary depends on the length of the input sequence. In this paper, we address this limitation by repurposing the attention mechanism of [5] to create pointers to input elements. We show that the resulting architecture, which we name Pointer Networks (Ptr-Nets), can be trained to output satisfactory solutions to three combinatorial optimization problems – computing planar convex hulls, Delaunay triangulations and the symmetric planar Travelling Salesman Problem (TSP). The resulting models produce approximate solutions to these problems in a purely data driven fashion.

*Equal contribution

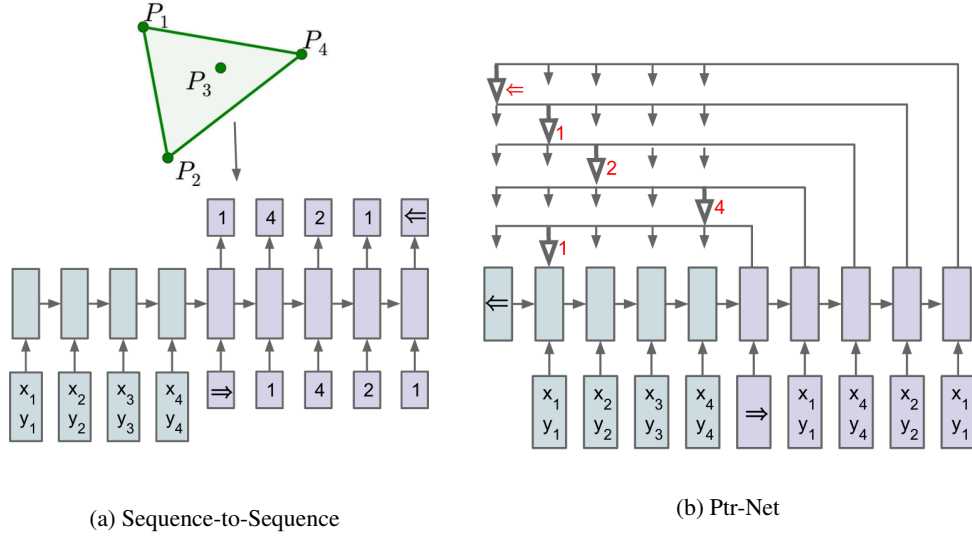


Figure 1: **(a) Sequence-to-Sequence** - An RNN (blue) processes the input sequence to create a code vector that is used to generate the output sequence (purple) using the probability chain rule and another RNN. The output dimensionality is fixed by the dimensionality of the problem and it is the same during training and inference [1]. **(b) Ptr-Net** - An encoding RNN converts the input sequence to a code (blue) that is fed to the generating network (purple). At each step, the generating network produces a vector that modulates a content-based attention mechanism over inputs ([5, 2]). The output of the attention mechanism is a softmax distribution with dictionary size equal to the length of the input.

ion (i.e., when we only have examples of inputs and desired outputs). The proposed approach is depicted in Figure 1.

The main contributions of our work are as follows:

- We propose a new architecture, that we call Pointer Net, which is simple and effective. It deals with the fundamental problem of representing variable length dictionaries by using a softmax probability distribution as a “pointer”.
- We apply the Pointer Net model to three distinct non-trivial algorithmic problems involving geometry. We show that the learned model generalizes to test problems with more points than the training problems.
- Our Pointer Net model learns a competitive small scale ($n \leq 50$) TSP approximate solver. Our results demonstrate that a purely data driven approach can learn approximate solutions to problems that are computationally intractable.

2 Models

We review the sequence-to-sequence [1] and input-attention models [5] that are the baselines for this work in Sections 2.1 and 2.2. We then describe our model - Ptr-Net in Section 2.3.

2.1 Sequence-to-Sequence Model

Given a training pair, $(\mathcal{P}, \mathcal{C}^{\mathcal{P}})$, the sequence-to-sequence model computes the conditional probability $p(\mathcal{C}^{\mathcal{P}}|\mathcal{P}; \theta)$ using a parametric model (an RNN with parameters θ) to estimate the terms of the probability chain rule (also see Figure 1), i.e.

$$p(\mathcal{C}^{\mathcal{P}}|\mathcal{P}; \theta) = \prod_{i=1}^{m(\mathcal{P})} p_{\theta}(C_i|C_1, \dots, C_{i-1}, \mathcal{P}; \theta). \quad (1)$$

Here $\mathcal{P} = \{P_1, \dots, P_n\}$ is a sequence of n vectors and $\mathcal{C}^{\mathcal{P}} = \{C_1, \dots, C_{m(\mathcal{P})}\}$ is a sequence of $m(\mathcal{P})$ indices, each between 1 and n .

The parameters of the model are learnt by maximizing the conditional probabilities for the training set, i.e.

$$\theta^* = \arg \max_{\theta} \sum_{\mathcal{P}, \mathcal{C}^{\mathcal{P}}} \log p(\mathcal{C}^{\mathcal{P}} | \mathcal{P}; \theta), \quad (2)$$

where the sum is over training examples.

As in [1], we use an Long Short Term Memory (LSTM) [11] to model $p_{\theta}(C_i | C_1, \dots, C_{i-1}, \mathcal{P}; \theta)$. The RNN is fed P_i at each time step, i , until the end of the input sequence is reached, at which time a special symbol, $\Rightarrow$ is input to the model. The model then switches to the generation mode until the network encounters the special symbol $\Leftarrow$, which represents termination of the output sequence.

Note that this model makes no statistical independence assumptions. We use two separate RNNs (one to encode the sequence of vectors P_j , and another one to produce or decode the output symbols C_i). We call the former RNN the encoder and the latter the decoder or the generative RNN.

During inference, given a sequence $\mathcal{P}$, the learnt parameters θ^* are used to select the sequence $\hat{\mathcal{C}}^{\mathcal{P}}$ with the highest probability, i.e., $\hat{\mathcal{C}}^{\mathcal{P}} = \arg \max_{\mathcal{C}^{\mathcal{P}}} p(\mathcal{C}^{\mathcal{P}} | \mathcal{P}; \theta^*)$. Finding the optimal sequence $\hat{\mathcal{C}}$ is computationally impractical because of the combinatorial number of possible output sequences. Instead we use a beam search procedure to find the best possible sequence given a beam size.

In this sequence-to-sequence model, the output dictionary size for all symbols C_i is fixed and equal to n , since the outputs are chosen from the input. Thus, we need to train a separate model for each n . This prevents us from learning solutions to problems that have an output dictionary with a size that depends on the input sequence length.

Under the assumption that the number of outputs is $O(n)$ this model has computational complexity of $O(n)$. However, exact algorithms for the problems we are dealing with are more costly. For example, the convex hull problem has complexity $O(n \log n)$. The attention mechanism (see Section 2.2) adds more “computational capacity” to this model.

2.2 Content Based Input Attention

The vanilla sequence-to-sequence model produces the entire output sequence $\mathcal{C}^{\mathcal{P}}$ using the fixed dimensional state of the recognition RNN at the end of the input sequence $\mathcal{P}$. This constrains the amount of information and computation that can flow through to the generative model. The attention model of [5] ameliorates this problem by augmenting the encoder and decoder RNNs with an additional neural network that uses an attention mechanism over the entire sequence of encoder RNN states.

For notation purposes, let us define the encoder and decoder hidden states as $(e_1, \dots, e_n)$ and $(d_1, \dots, d_{m(\mathcal{P})})$, respectively. For the LSTM RNNs, we use the state after the output gate has been component-wise multiplied by the cell activations. We compute the attention vector at each output time i as follows:

$$\begin{aligned} u_j^i &= v^T \tanh(W_1 e_j + W_2 d_i) & j \in (1, \dots, n) \\ a_j^i &= \text{softmax}(u_j^i) & j \in (1, \dots, n) \\ d'_i &= \sum_{j=1}^n a_j^i e_j \end{aligned} \quad (3)$$

where softmax normalizes the vector u^i (of length n) to be the “attention” mask over the inputs, and v , W_1 , and W_2 are learnable parameters of the model. In all our experiments, we use the same hidden dimensionality at the encoder and decoder (typically 512), so v is a vector and W_1 and W_2 are square matrices. Lastly, d'_i and d_i are concatenated and used as the hidden states from which we make predictions and which we feed to the next time step in the recurrent model.

Note that for each output we have to perform n operations, so the computational complexity at inference time becomes $O(n^2)$.